SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

SIMATS ENGINEERING
CSA17 – ARTIFICIAL INTELLIGENCE
ASSESSMENT TOOL 1: SCENARIO-BASED ASSIGNMENT – ANSWERS

Name: ARAVINDH R Reg. No.: 192312024

1. AI-Powered Emergency Medicine Delivery Drone

i. Greedy Best-First Search

Greedy Best-First Search selects the frontier location with the smallest heuristic h(n), here the straight-line distance to the delivery point. It can make quick decisions and explore fewer nodes, which is useful during emergencies. However, it ignores the cost already travelled, may choose difficult terrain or a weather-affected route, and can be misled by incomplete information. It does not guarantee the lowest-cost or safest route.

ii. A* Search

A* uses f(n) = g(n) + h(n), where g(n) is the known cost from the start and h(n) estimates the remaining cost. The accumulated cost can represent terrain and weather delays, while the heuristic guides the drone toward the destination. With non-negative costs and an admissible heuristic, A* finds an optimal route in a static, correctly modelled environment. Straight-line distance is useful if it does not overestimate the true remaining cost.

iii. Heuristic accuracy

A good heuristic directs search toward promising routes. An inaccurate heuristic can mislead Greedy Search; if it overestimates the remaining cost, it can also remove A*'s optimality guarantee. A heuristic that underestimates the cost is generally safe for A* but may cause more exploration. Weather and terrain should be reflected in updated route-cost estimates.

iv. Dynamic blocked paths

When a path becomes blocked, the drone must mark it unavailable and replan from its current position. Repeated changes can make a previously calculated route obsolete and increase computation. Frequent sensing and incremental replanning can reuse previous search work and reduce delay.

v. Recommendation

A* with real-time updates and replanning is the best general choice because it balances actual travel cost with estimated remaining cost. For strict deadlines, a bounded or incremental A* variant can be used. The drone must validate the next route segment against current obstacle and weather information; if safety cannot be confirmed, it should pause or select a verified fallback route.

2. Smart-City Traffic-Signal Optimization

Formulation

Let variables represent each signal's green time, phase order and offset. Minimize $J = \alpha W + \beta F + \gamma C$, where W is total waiting time, F is fuel consumption and C is congestion; α, β and γ represent the city's priorities. Constraints include safe phase transitions, minimum/maximum green times, pedestrian crossing time, and intersection capacity. Traffic data updates the objective continuously.

i. Hill Climbing

Hill Climbing starts with a signal plan and repeatedly tests neighbouring plans, keeping a change when it improves the objective. It is simple and fast, but may stop at a local optimum, depend on the starting plan, or fail to find improvements requiring coordinated changes. Exhaustive traditional search is inefficient because combinations across hundreds of intersections grow extremely quickly.

ii. Local maxima, plateaus and ridges

A local optimum occurs when nearby changes do not improve the plan although a better distant plan exists. A plateau is a region where many plans have similar scores, giving little guidance. A ridge occurs when improvement requires changing several signals together; changing only one signal may appear unhelpful even though coordinated offsets could reduce queues.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing sometimes accepts a worse solution early, helping it escape local optima; this probability decreases as its temperature parameter cools. A Genetic Algorithm keeps a population of plans and uses selection, crossover and mutation to explore alternatives. Both can search beyond immediate neighbours, but need tuning and computation. Unsafe plans must always be rejected.

iv. Recommendation

A hybrid approach is suitable: use a Genetic Algorithm or Simulated Annealing in a rolling optimization window to find coordinated plans, then use a fast controller for safe real-time adjustments. Genetic Algorithms suit joint optimization across many intersections; Simulated Annealing may be simpler to implement. Compare approaches using delay, queue length, fuel use, computation time and safety compliance.

3. Mars Rover: Online Search

i. Why online search is needed

Offline search assumes a sufficiently complete model before acting. A Mars rover explores unknown terrain and discovers obstacles as it moves. An online agent alternates between sensing, acting, learning and replanning, so each next action can use newly discovered information.

ii. Partial observability and dynamic challenges

Sensors have limited range and may be noisy, so the rover cannot know every hazard in advance. Rocks or craters may be hidden, and communication delays can prevent immediate instructions. The rover should represent uncertainty, move cautiously, avoid risky unknown areas, and revise plans when observations change.

iii. Internal model

The rover combines sensor observations with its estimated position to update a local map. It records known free areas, obstacles, uncertain regions and estimated travel costs. After each action, it compares expected and observed results, corrects its map and position estimate, and replans if the route is blocked.

iv. Comparison of search strategies

Uninformed methods such as Breadth-First Search and Uniform-Cost Search use little or no goal-direction information and may explore many locations. Informed methods such as Greedy Best-First Search and A* use heuristics to prioritize promising locations, but a complete fixed map may not be available. Online search adds a sense–act–learn cycle and can use either informed or uninformed planning over the known map.

v. Recommendation

Use online, receding-horizon planning with A*-style heuristic guidance over the known local map. Execute a safe next action, update the map using sensors, and replan. Assign risk to unknown terrain and keep a safe fallback or stop when uncertainty becomes unacceptable. This balances adaptability, efficient exploration and safety.

4. University Examination Timetabling as a CSP

i. Variables, domains and constraints

Variables: each exam's time slot and hall, and where necessary an invigilator. Domains: available time slots and halls that meet capacity, accessibility and equipment needs. Hard constraints: students cannot have overlapping exams; a hall cannot host two exams simultaneously; hall capacity must be sufficient; prerequisite scheduling order must be respected; faculty must be available; and approved accommodations must be met. Soft constraints may minimize long gaps and balance hall use.

ii. Backtracking

Backtracking assigns a time and hall to one exam at a time and checks constraints after each assignment. If the assignment is valid, it proceeds. If no legal value remains for an exam, it returns to an earlier exam, removes that assignment and tries another value. A complete assignment satisfying every hard constraint is a valid timetable.

iii. Constraint propagation

Forward checking removes incompatible values from the domains of unassigned exams after each assignment. For example, scheduling one exam removes that time slot from exams sharing students and removes an occupied hall from other exams at that time. If any variable has no legal value left, failure is detected early and the algorithm backtracks.

iv. Scalability and strategy

The problem grows with the number of courses, students, halls and constraints, and enrolments or faculty availability may change. Combine propagation with minimum-remaining-values (choose the exam with fewest legal options), degree heuristic (choose the exam constraining most others), and least-constraining-value ordering. Large cases can be decomposed into blocks and repaired locally after changes. Verify all hard constraints before publishing.

5. Strategic Game AI: Minimax and Alpha-Beta Pruning

i. Minimax model

Minimax models two adversarial players. MAX chooses moves to maximize the score, while MIN chooses moves to minimize it. The algorithm searches possible moves to a chosen depth, evaluates terminal or cutoff states, then propagates values upward: MAX takes the largest child value and MIN takes the smallest. The AI selects the move with the best worst-case outcome, assuming an optimal opponent.

ii. Computational challenges

With branching factor b and depth d, basic Minimax examines approximately O(b^d) nodes. A large tree can exceed time and memory limits. A shallow search may miss long-term effects, while a deep search may not finish in time. A useful evaluation function is needed when search stops before a terminal state.

iii. Alpha-Beta pruning

Alpha-Beta pruning produces the same decision as Minimax for the searched tree but skips branches that cannot change the result. Alpha (α) is MAX's best guaranteed value so far; beta (β) is MIN's best guaranteed value. MAX updates α with the largest child value and MIN updates β with the smallest. When $\alpha \geq \beta$, remaining children can be pruned because that branch cannot improve the decision. Good move ordering improves pruning; best-case complexity approaches O(b^(d/2)), while worst-case complexity remains O(b^d).

iv. Evaluation function

An illustrative function is $E(s) = 5M + 3O + 2R + 4P - 6H$, where M is resource/material advantage, O is objective control, R is resource income, P is positional advantage and H is immediate threat to the AI. The weights are

examples and must be tuned to the game. Terminal wins and losses should receive much larger positive and negative scores than ordinary states.

v. Real-time recommendation

Use depth-limited Minimax with Alpha-Beta pruning, good move ordering and a fast evaluation function. Iterative deepening searches depth 1, then 2, then 3, retaining the best completed result if time expires. A transposition table can reuse evaluations of repeated states. Return the best move from the deepest completed search rather than waiting for an unfinished search.

Conclusion

The appropriate AI technique depends on the environment and objective: A* with replanning supports cost-sensitive routing; metaheuristics explore large traffic-optimization spaces; online search adapts to unknown terrain; CSP methods enforce timetable constraints; and Minimax with Alpha-Beta pruning supports adversarial decisions under time limits. Current data, safety checks and performance evaluation are essential in every case.

Reference note

Answers are based on the five scenarios and tasks in the supplied assessment sheet. Where the assessment provides no numerical data, graph layout or game-specific rules, explanations remain conceptual; the sample evaluation-function weights are illustrative.